

UVM Test, Environment, Sequencers & Sequences

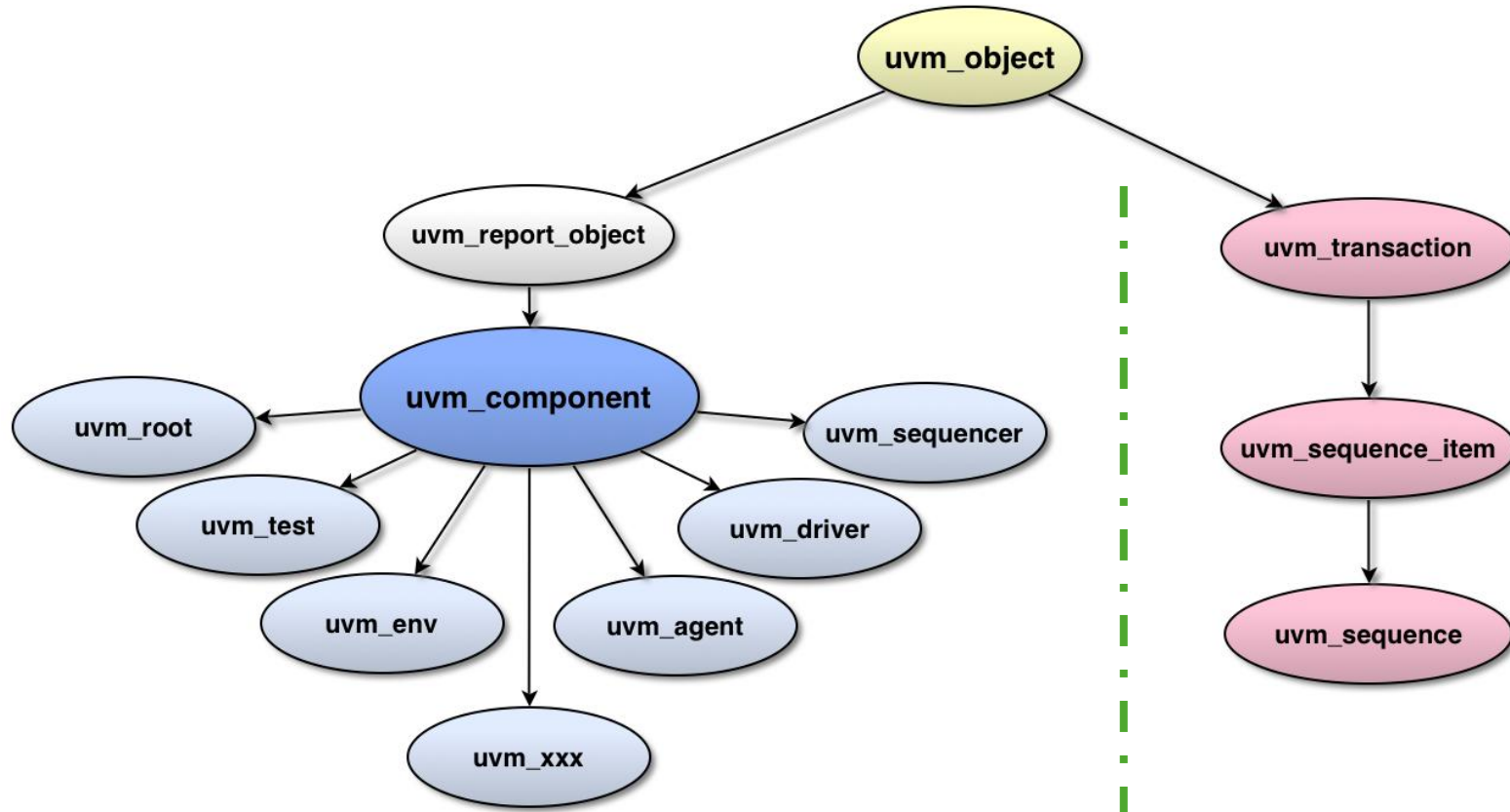
SyoSil, 2026

Contents

- Quick recap/Intro
- Test Class (uvm_test)
- Env Class (uvm_env)
- Virtual Sequencer/Sequencer (uvm_sequencer)
- Sequences (uvm_sequence)

Quick Recap/Intro

Base Classes



Structural component base classes

Transaction item and stimulus
generation base classes

UVM component - (uvm_component)

- UVM base library class
- Used for static testbench structure
- May only be created at the start of simulation
- Once created, present throughout simulation
- Are **PHASED!**
- Examples
 - `uvm_test`
 - `uvm_env`
 - `uvm_driver`
 - `uvm_agent`
 - `uvm_monitor`

Test (uvm_test)

Tests (uvm_test)

- Root of the UVM test bench hierarchy
 - For the user (uvm_root is the real root)
- Instantiates
 - Environment configuration
 - Environment
 - Connects top level module interfaces with environment
- Typically
 - Starts clocks
 - Releases reset
 - Starts a top level sequence

```
class my_base_test(uvm_test):
    def __init__(self, name="my_test", parent=None):
        super().__init__(name, parent)

        # Configuration object handler
        self.cfg = None

        # Declare Environment handler
        self.env = None

    def build_phase(self):
        super().build_phase()

        # Create configuration objects
        self.cfg = tb_config.create('cfg')

        # Instantiate Environment
        self.env = tb_env.create("env", self)
        self.env.cdb_set("cfg", self.cfg, "")
```

Example of a Base Test and Actual Test

```
from pyuvvm import *
```

```
class base_test(uvm_test):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)
        self.clk_interface = None
        self.cfg = None
        self.environment = None

    def build_phase(self, phase):
        super().build_phase(phase)

        # Get clk interface from config_db
        if not uvm_config_db.get(self, "", "clk_if", self.clk_interface):
            uvm_fatal("BASE_TEST", "No virtual clk_if passed to base_test")

        # Create configuration object
        self.cfg = cfg("cfg")
        self.cfg.is_active = uvm_active_passive_enum.UVM_ACTIVE

        # Set configuration object in config_db
        uvm_config_db.set(self, "env", "cfg", self.cfg)

        # Create environment
        self.environment = env("env", self)

    async def run_phase(self, phase):
        phase.raise_objection(self)

        await super().run_phase(phase)

        # Wait until rst is deasserted
        while not self.rst_interface.rst == 0:
            await Timer(1, "NS") # Adjust the time unit as needed

        phase.drop_objection(self)
```

Creates the ENV cfg
and ENV

Send the ENV cfg to
the ENV via the
ConfigDB

```
from pyuvvm import *
```

```
class mft_test(base_test):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)

    async def run_phase(self, phase):
        phase.raise_objection(self)

        await super().run_phase(phase)

        # Create and randomize the mft_seq
        mft_seq_i = mft_seq("mft_seq")

        if not mft_seq_i.randomize():
            uvm_fatal("MFT_TEST", "Randomization failed")

        # Start the sequence
        await mft_seq_i.start(self.environment.vseqr, None)

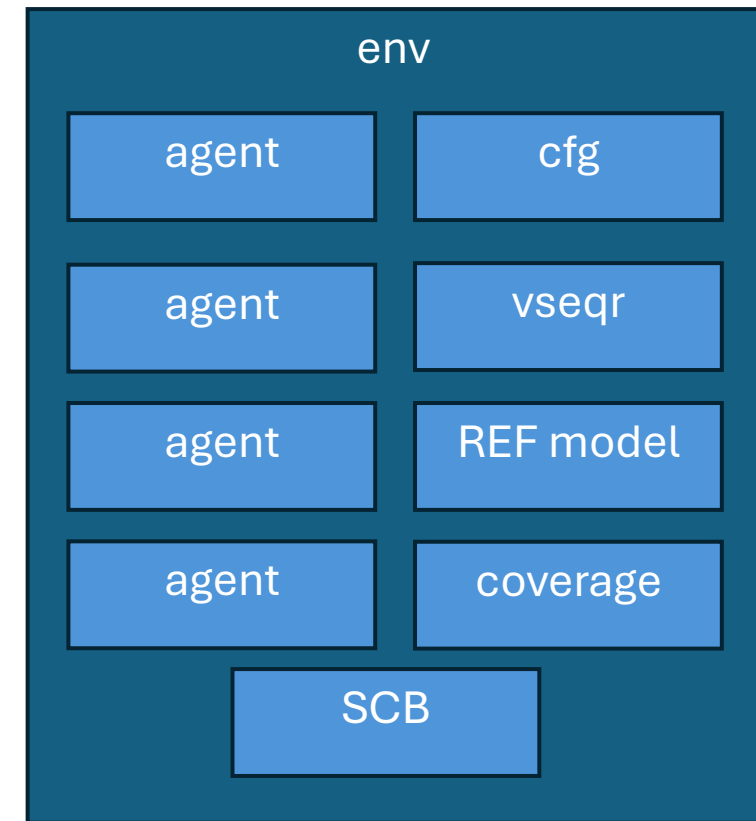
        phase.drop_objection(self)
```

Creates, randomizes and starts
the virtual sequence on the virtual
sequencer in the ENV

Environment (uvm_env)

Environment Class (uvm_env)

- UVM Component
- Instantiates UVCs, topology typically similar to DUT
- Container for cross-UVC testbench functionality
 - Virtual sequencer
 - Reference model
 - Coverage
- Hierarchical → May contain other environments
- Primary unit of re-use from block to system testbenches



Example of an Environment

```
class env(uvm env):
    def __init__(self, name, parent):
        super().__init__(name, parent)

        # Declare configuration object
        self.cfg = None

        # Declare Virtual Sequencer
        self.virtual_sequencer = None

        # Declare Scoreboard
        self.scb = None

    def build_phase(self):
        super().build_phase()

        # Instantiate configuration object
        self.cfg = self.cdb_get("cfg")

        # Instantiate Virtual Sequencer
        self.virtual_sequencer = vsegr.create("virtual_sequencer", self)
        self.virtual_sequencer.cdb_set("cfg", self.cfg, "")

        # Instantiate Scoreboard
        ...

        # SSDT Producer
        self.uvc_ssdt_producer = uvc_ssdt_agent.create("uvc_ssdt_producer", self)
        self.uvc_ssdt_producer.cdb_set("cfg", self.cfg.ssd_t_prod_cfg, "")
        self.cfg.ssd_t_prod_cfg.is_active = uvm_active_passive_enum.UVM_ACTIVE
        self.cfg.ssd_t_prod_cfg.driver_type = uvc_ssdt_type_enum.PRODUCER

    def connect_phase(self):
        super().connect_phase()

        # Connect agents' sequencer to virtual sequencer
        self.virtual_sequencer.ssd_t_producer_sequencer = self.uvc_ssdt_producer.sequencer
```

Get the ENV cfg from the test via the ConfigDB

Forward the ENV cfg to the virtual sequencer via the ConfigDB

Create the virtual sequencer

Connect the SSDT UVC's sequencer to the virtual sequencer

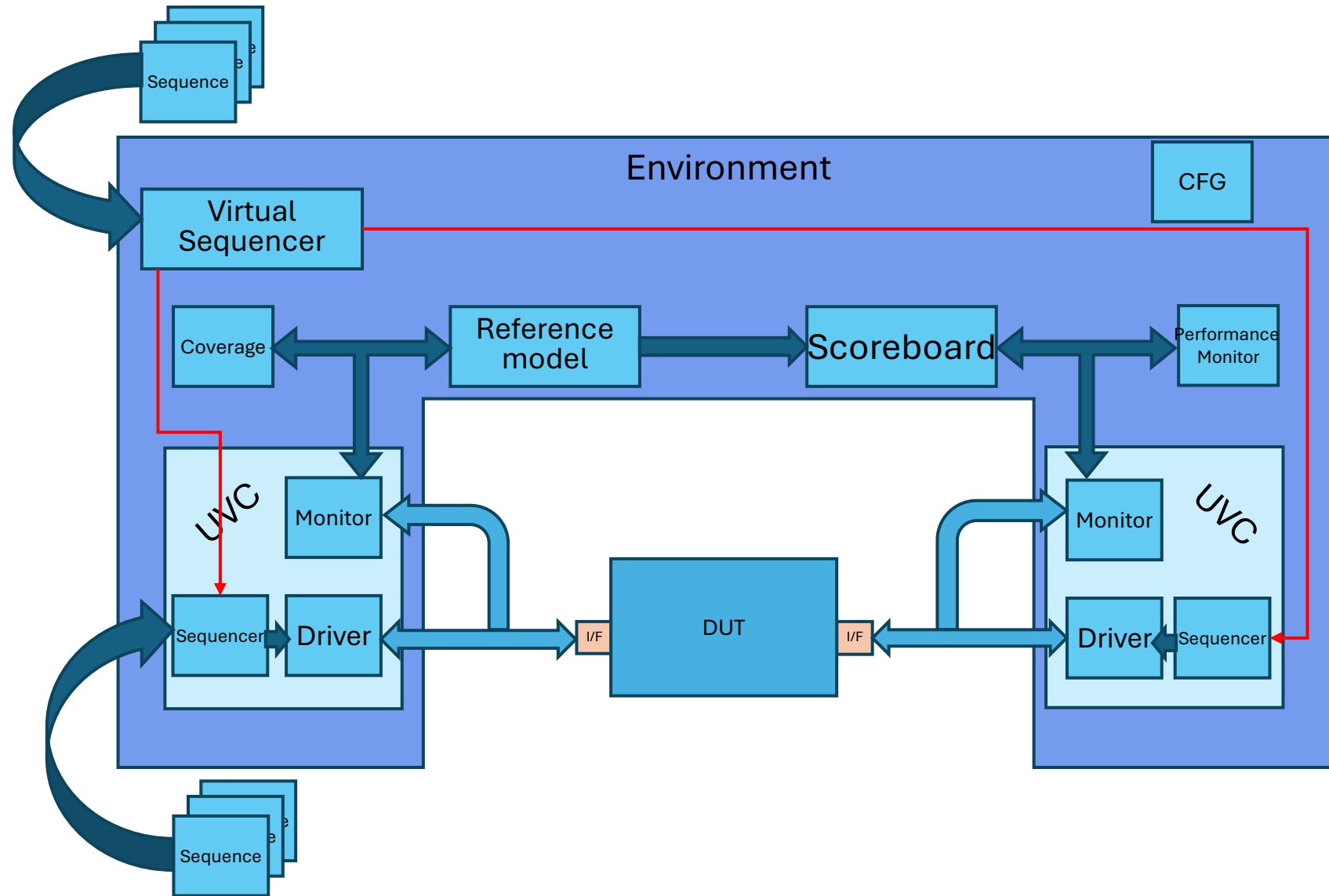
Creates the SSDT UVC

Sequencer (uvm_sequencer)

Environment Sequencer (uvm_sequencer)

- UVM Component
- Called 'virtual sequencer' in UVM terminology if not instantiated in a UVC
- Coordinates sequences that access multiple UVCs
 - In order to promote (vertical) reuse no sequences should refer directly to UVC sequencers. Instead use the virtual sequencer.
- Environment sequences are DUT specific but can often be re-used at system level

UVM Environment with Virtual Sequencer

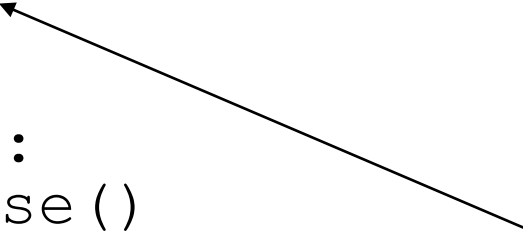


Example of a Virtual Sequencer

```
from pyuvvm import uvm_sequencer

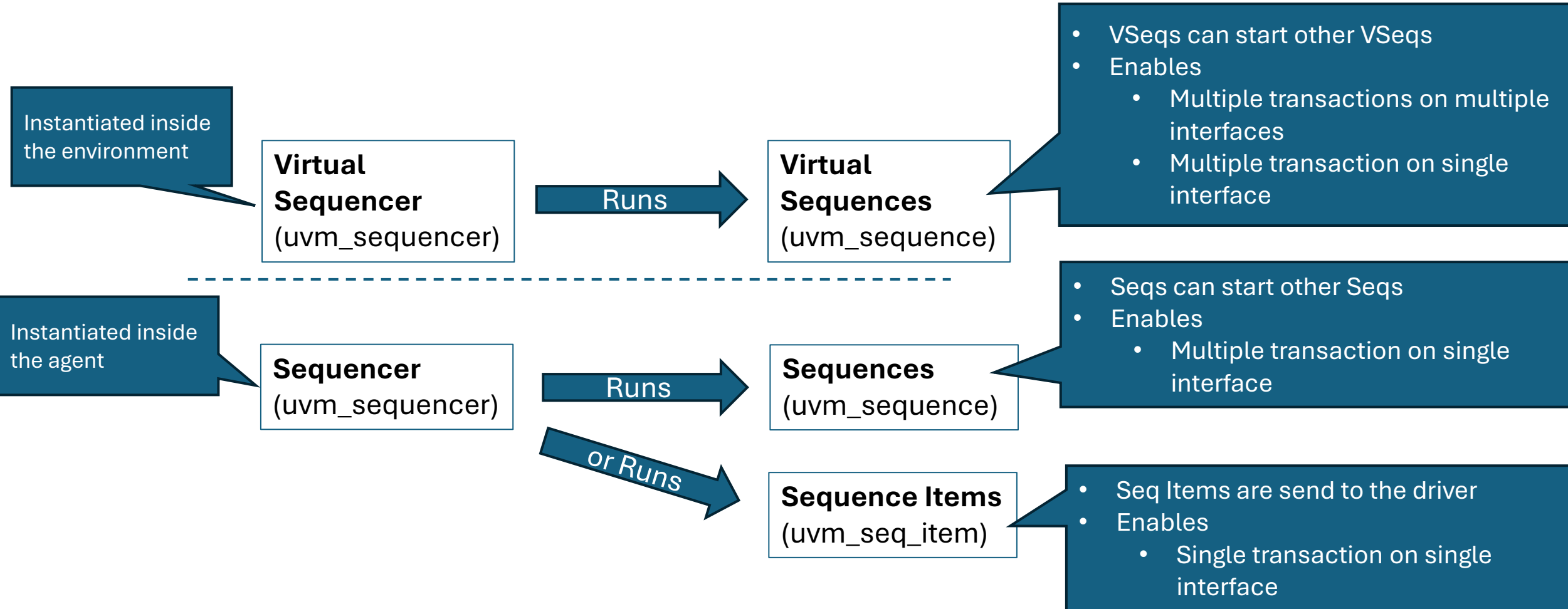
class virtual_sequencer(uvm_sequencer):
    def __init__(self, name="virtual_sequencer", parent=None):
        super().__init__(name, parent)
        # Declaration of components
        self.cfg = None          # Configuration object
        # Handler for producer sequencer
        self.ssd_t_producer_sequencer = None
        ...

    def build_phase(self):
        super().build_phase()
```



SSDT UVC Sequencer
Set by ENV (Parent) in the connect
phase

Virtual Sequencer, Virtual Sequences Sequencer, Sequences, Sequence Items



Virtual Sequences (uvm_sequence)

Virtual Sequences

- Derived from `uvm_sequence`
- Runs typically on a virtual sequencer instantiated in the env
- May start other sequences
 - Any nesting is allowed
 - On other sequencers if running on virtual sequencer
- Primary object for stimuli reuse
- Used for protocol layering and for stimuli that interacts with more than one interface/UVC

Virtual Sequences

- You create a virtual sequence by inheriting from the base class `uvm_sequence`
 - Key properties and methods that are inherited
 - `start()`
 - Method that starts the sequence execution
 - `body()`
 - Method that contains the body of the sequence, e.g:
 - Stimulus generation
 - Starting other (virtual) sequences

```
class simple_test(uvm_test):
    def __init__(self, name, parent=None):
        super().__init__(name, parent)

    async def run_phase(self, phase):
        await super().run_phase(phase)
        phase.raise_objection(self)

        # Create and start the virtual sequence
        virtual_seq = simple_virtual_seq()

        # Start on the virtual sequencer
        await virtual_seq.start(self.virtual_sequencer)
        phase.drop_objection(self)

# Define a virtual sequence
class simple_virtual_seq(uvm_sequence):
    def __init__(self, name="simple_virtual_seq"):
        super().__init__(name)

    async def body(self):
        ...
```

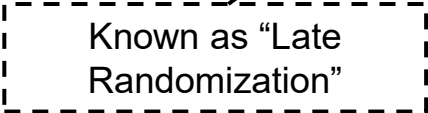
Creates and starts the virtual sequence on the virtual sequencer in the ENV

Sequence (uvm_sequence)

Sequence (Running on Sequencer in UVC)

- Dual API
- API1 - Sequence API
 - For starting other sequences – start()
- API2 - Sequence Item API
 - Flow for generating a sequence item in sequence
 - **Create** the sequence item
 - Call **start_item()** – Blocks until “picked” by sequencer
 - **Randomize** the sequence item
 - Call **finish_item()**

Known as “Late
Randomization”



```
class simple_seq(uvm_sequence):
    def __init__(self, name="simple_seq"):
        super().__init__(name)

    async def body(self):
        # Create a sequence item
        seq_item = simple_seq_item()

        # Call start_item await
        self.start_item(seq_item)

        # Randomize the sequence
        item if not seq_item.randomize():
            uvm_fatal("SIMPLE_SEQ",
                    "Randomization failed")

        # Call finish_item
        self.finish_item(seq_item)

        # Print the randomized
        item uvm_info("SIMPLE_SEQ",
            f"Randomized seq_item: {seq_item}",
            UVM_LOW)
```

What Can This Structure Do?

Feature	
UVM Test with config change 01	1. Derive config and change config 2. Add factory overwrite to config creation in derived test
UVM Test with config change 02	Change config in derived test after super.build_phase
UVM Test with new top level virtual sequence	1. Implement new virtual sequence derived from base virtual sequence 2. Add factory overwrite to virtual sequence derived test for the new virtual sequence
UVM Test with any virtual sequence/sequence/sequence item replaced	1. Implement derived versions of what ever you need to have changed 2. Add factory overwrites for all in the derived test
UVM test with uvm_components being replaced (rare thing to do)	1. Implement derived version of what ever you need to have changed 2. Add factory overwrites for all in the derived test

A Tool to Help: PyUVMTBViewer

- Located in lectures/lessons/L08_UVM_test_env_seqs
- PyUVMTBViewer
- Displays
 - The uvm_component hierarchy
 - The phases
 - Still a prototype
 - Should work on any PyUVM testbench
- Run
 - `cd in lectures/lessons`
 - `python -m PyUVMTBViewer`
 - Select open file and navigate to JSON input file